



Övning 3: Dekomposition och dynamisk programmering

DD2350 Algoritmer, datastrukturer och komplexitet

Marcus Dicander

EECS / Teoretisk datalogi, KTH

HT 2026

- 1 Teori: rekursion och dekomposition
- 2 Uppgift 1–4: dekomposition
- 3 Teori: dynamisk programmering
- 4 Uppgift 5–6: dynamisk programmering

Uppgifter

- 1–4: dekomposition
- 5–6: dynamisk programmering

Uppgifter och lösningsförslag: Viggo Kann.

Teori: rekursion och dekomposition

Rekursion

- *Rekursion* handlar om saker som refererar till eller beror av sig själva.
- Exempel: "Om du inte förstår rekursion, läs den här meningen igen."
- Rekursiva akronymer: **GNU** = GNU's Not Unix, **WINE** = Wine Is Not an Emulator, **PHP** = PHP: Hypertext Preprocessor.
- Prova att söka på ordet:

rekursion



Menade du: *rekursion*

Rekursion i matematiken

Definition: rekursiv funktion

En rekursiv funktion är en funktion som definieras i termer av sig själv.

$$n! = \begin{cases} 1 & \text{om } n = 0, \\ n \cdot (n-1)! & \text{annars.} \end{cases}$$

$$F_n = \begin{cases} 0 & \text{om } n = 0, \\ 1 & \text{om } n = 1, \\ F_{n-1} + F_{n-2} & \text{annars.} \end{cases}$$



basfallet nås

Viktigt!

Se till att ha heltäckande basfall, och att varje funktionsevaluering kan reduceras till dessa.

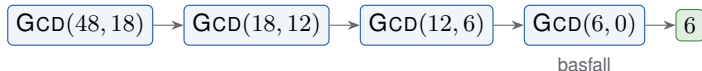
Rekursion i datalogin

Definition: rekursiv algoritm

En rekursiv algoritm är en algoritm som anropar sig själv.

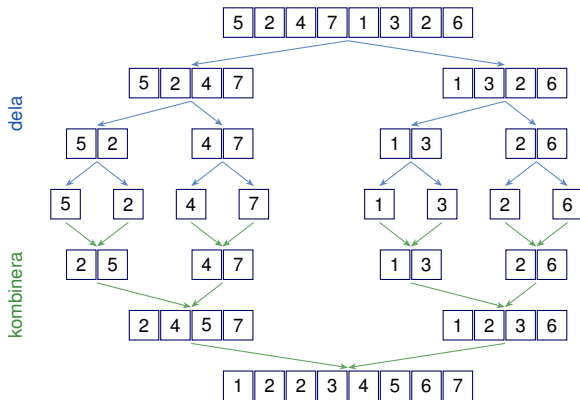
```
1: function GCD( $a, b$ )  
2:   if  $b = 0$  then return  $a$   
3:   return GCD( $b, a \bmod b$ )
```

- Andra exempel: mergesort, fakultet, Fibonaccital.
- Även här: heltäckande basfall!



Dekomposition (divide and conquer)

- Metod för algoritmkonstruktion som bygger på rekursion.
- Idé:
 1. **Dela** upp i mindre delproblem.
 2. **Lös** delproblemen rekursivt.
 3. **Kombinera** dellösningarna.
- Glöm inte basfallen!
- Klassiskt exempel: mergesort.



Mästarsatsen

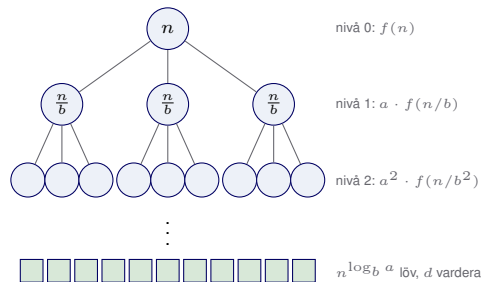
Sats

Låt $a \geq 1$, $b > 1$, $d > 0$ och

$$T(n) = aT\left(\frac{n}{b}\right) + f(n), \quad T(1) = d.$$

Då gäller

1. $T(n) \in \Theta(n^{\log_b a})$ om $f(n) \in O(n^{\log_b a - \varepsilon})$, $\varepsilon > 0$
2. $T(n) \in \Theta(n^{\log_b a} \log n)$ om $f(n) \in \Theta(n^{\log_b a})$
3. $T(n) \in \Theta(f(n))$ om $f(n) \in \Omega(n^{\log_b a + \varepsilon})$, $\varepsilon > 0$



Intuition från rekursionsträdet:

1. löven dominerar arbetet
2. alla $\log_b n$ nivåer bidrar lika mycket
3. roten dominerar

Uppgift 1–4: dekomposition

Uppgift 1: Max och min med dekomposition

Uppgift

I vektorn $v[1..n]$ ligger n tal. Konstruera en dekompositionsalgoritm som tar reda på det största och det minsta talet i v . Algoritmen ska använda högst $\lceil 3n/2 \rceil - 2$ jämförelser mellan v -element. Antalet tal i v behöver inte vara en tvåpotens.

Uppgift 1: Max och min med dekomposition

Uppgift

I vektorn $v[1..n]$ ligger n tal. Konstruera en dekompositionsalgoritm som tar reda på det största och det minsta talet i v . Algoritmen ska använda högst $\lceil 3n/2 \rceil - 2$ jämförelser mellan v -element. Antalet tal i v behöver inte vara en tvåpotens.

- Naivt: linjärsök efter max ($n - 1$ jämförelser) och sedan efter min ($n - 1$): totalt $2n - 2$.
- Målet $\lceil 3n/2 \rceil - 2 \approx 1,5n$ är alltså en fjärdedel färre jämförelser.

Uppgift 1: Max och min med dekomposition

Uppgift

I vektorn $v[1..n]$ ligger n tal. Konstruera en dekompositionsalgoritm som tar reda på det största och det minsta talet i v . Algoritmen ska använda högst $\lceil 3n/2 \rceil - 2$ jämförelser mellan v -element. Antalet tal i v behöver inte vara en tvåpotens.

- Naivt: linjärsök efter max ($n - 1$ jämförelser) och sedan efter min ($n - 1$): totalt $2n - 2$.
- Målet $\lceil 3n/2 \rceil - 2 \approx 1,5n$ är alltså en fjärdedel färre jämförelser.
- Nyckelidé: jämför två element med *varandra*. Det större kan bara bli max, det mindre bara min. Ett par kostar en jämförelse men ger både en max- och en minkandidat.

Uppgift 1: dekompositionsstegen

1. **Dela:** dela vektorn i två delar.
2. **Lös rekursivt:** anropa algoritmen på varje del och få $(\min_1, \max_1)$ och $(\min_2, \max_2)$.
3. **Kombinera:** $\max = \max(\max_1, \max_2)$ och $\min = \min(\min_1, \min_2)$ – två jämförelser.

Basfall:

- ett element: returnera $(v[i], v[i])$ – noll jämförelser,
- två element: *en* jämförelse ger både min och max.

Uppgift 1: dekompositionsstegen

1. **Dela:** dela vektorn i två delar.
2. **Lös rekursivt:** anropa algoritmen på varje del och få $(\min_1, \max_1)$ och $(\min_2, \max_2)$.
3. **Kombinera:** $\max = \max(\max_1, \max_2)$ och $\min = \min(\min_1, \min_2)$ – två jämförelser.

Basfall:

- ett element: returnera $(v[i], v[i])$ – noll jämförelser,
- två element: *en* jämförelse ger både min och max.

Delningen måste göras med omsorg

Två ensamma element som kombineras kostar $0 + 0 + 2 = 2$ jämförelser, men ett par kostar bara 1. Vi vill därför att så många löv som möjligt är par.

Uppgift 1: algoritmen

```
1: function MINMAX( $v, i, j$ )
2:   if  $i = j$  then return ( $v[i], v[i]$ )
3:   if  $i + 1 = j$  then
4:     if  $v[i] < v[j]$  then return ( $v[i], v[j]$ )
5:     else return ( $v[j], v[i]$ )
6:    $m \leftarrow \lfloor (j - i) / 2 \rfloor$ 
7:   if  $m$  är udda then  $m \leftarrow m + 1$ 
8:    $(min_1, max_1) \leftarrow \text{MINMAX}(v, i, i + m - 1)$ 
9:    $(min_2, max_2) \leftarrow \text{MINMAX}(v, i + m, j)$ 
10:  return ( $\min(min_1, min_2), \max(max_1, max_2)$ )
```

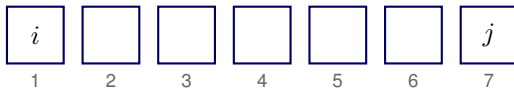
▷ *returnerar* ($\min, \max$) för $v[i..j]$

▷ *vänstra delen får jämnt antal element*

▷ *två jämförelser*

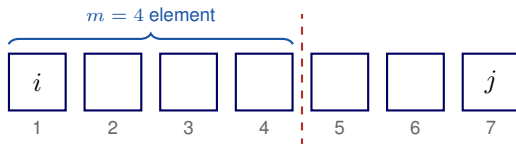
Tidskomplexitet: $T(n) = 2T(n/2) + O(1)$ ger $T(n) \in \Theta(n)$ (mästarsatsen, fall 1). Men uppgiften frågar efter det *exakta* antalet jämförelser, så vi räknar dem.

Uppgift 1: hur delas vektorn?



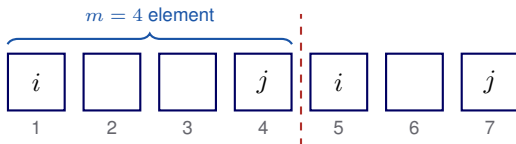
- Här är $n = j - i + 1 = 7$.

Uppgift 1: hur delas vektorn?



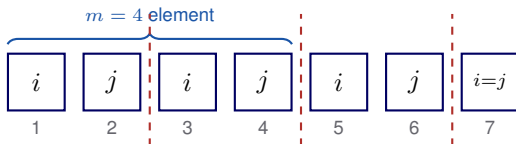
- Här är $n = j - i + 1 = 7$.
- $m \leftarrow \lfloor (j - i) / 2 \rfloor = 3$ är udda, så $m \leftarrow 4$: vänstra delen $v[i..i+m-1]$ får alltid ett *jämnt* antal element.

Uppgift 1: hur delas vektorn?



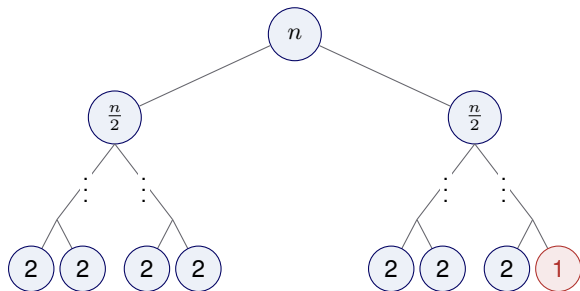
- Här är $n = j - i + 1 = 7$.
- $m \leftarrow \lfloor (j - i) / 2 \rfloor = 3$ är udda, så $m \leftarrow 4$: vänstra delen $v[i..i+m-1]$ får alltid ett *jämnt* antal element.
- Rekursiva anrop på $v[i..i+m-1]$ och $v[i+m..j]$. Högra delen är udda precis när n är udda.

Uppgift 1: hur delas vektorn?



- Här är $n = j - i + 1 = 7$.
- $m \leftarrow \lfloor (j - i) / 2 \rfloor = 3$ är udda, så $m \leftarrow 4$: vänstra delen $v[i..i+m-1]$ får alltid ett *jämnt* antal element.
- Rekursiva anrop på $v[i..i+m-1]$ och $v[i+m..j]$. Högra delen är udda precis när n är udda.
- Alla löv i rekursionsträdet blir därför par – utom möjligen det högraste, som är ett ensamt element om n är udda.

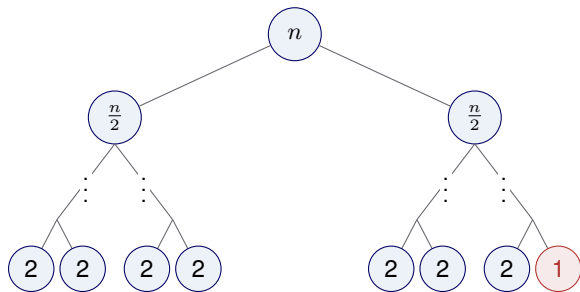
Uppgift 1: blir det verkligen $\lceil 3n/2 \rceil - 2$ jämförelser?



löv = basfall: par, samt ett ensamt element om n är udda

- Rekursionsträdet har $\lceil n/2 \rceil$ löv och därmed $\lceil n/2 \rceil - 1$ inre noder.
- Löven: ett par kostar 1 jämförelse, det ensamma elementet 0. Totalt $\lfloor n/2 \rfloor$.
- Inre noder: 2 jämförelser vardera. Totalt $2\lceil n/2 \rceil - 2$.
- Summa:
 $\lfloor n/2 \rfloor + 2\lceil n/2 \rceil - 2 = \lceil 3n/2 \rceil - 2$.

Uppgift 1: blir det verkligen $\lceil 3n/2 \rceil - 2$ jämförelser?



löv = basfall: par, samt ett ensamt element om n är udda

- Rekursionsträdet har $\lceil n/2 \rceil$ löv och därmed $\lceil n/2 \rceil - 1$ inre noder.
- Löven: ett par kostar 1 jämförelse, det ensamma elementet 0. Totalt $\lfloor n/2 \rfloor$.
- Inre noder: 2 jämförelser vardera. Totalt $2\lceil n/2 \rceil - 2$.
- Summa:
 $\lfloor n/2 \rfloor + 2\lceil n/2 \rceil - 2 = \lceil 3n/2 \rceil - 2$.

Går det bättre?

Nej: $\lceil 3n/2 \rceil - 2$ jämförelser är också en undre gräns (visas med ett motståndarargument).

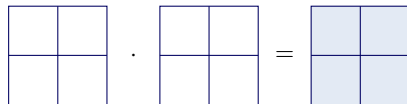
Uppgift 2: Matrimultiplikation

Strassens algoritm

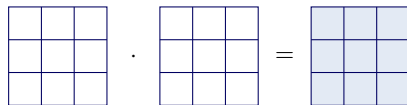
- Multiplicerar två $n \times n$ -matriser i tid $O(n^{2,808})$.
- Delar upp matriserna i 2×2 blockmatriser av storlek $\frac{n}{2} \times \frac{n}{2}$.
- Snabbare än $O(n^3)$ eftersom den gör *sju* blockmultiplikationer i stället för åtta.

Idé: dela i 3×3 blockmatriser i stället?

- Viggo försökte hitta det minimala antalet multiplikationer för två 3×3 -matriser och kom *nästan* ner till 22.



2 $\times$ 2 block: 8 blockprodukter naivt, Strassen klarar sig med 7.
 $T(n) = 7 T(n/2) + O(n^2) \Rightarrow \Theta(n^{\log_2 7}) \approx \Theta(n^{2,808})$



3 $\times$ 3 block: 27 blockprodukter naivt – Viggo nästan nere i 22.
 $T(n) = 22 T(n/3) + O(n^2) \Rightarrow ?$

Uppgift

Om det hade räckt med 22 rekursiva anrop, vilken tidskomplexitet hade det gett?

Uppgift 2: lösning

- Låt $T(n)$ vara tiden för att multiplicera två $n \times n$ -matriser.
- 22 rekursiva anrop, vart och ett på $\frac{n}{3} \times \frac{n}{3}$ -block: $22 T(n/3)$.
- Att kombinera blocken är bara additioner – linjärt i antalet element, alltså $O(n^2)$.

$$T(n) = 22 T\left(\frac{n}{3}\right) + O(n^2), \quad T(1) = O(1).$$

Uppgift 2: lösning

- Låt $T(n)$ vara tiden för att multiplicera två $n \times n$ -matriser.
- 22 rekursiva anrop, vart och ett på $\frac{n}{3} \times \frac{n}{3}$ -block: $22 T(n/3)$.
- Att kombinera blocken är bara additioner – linjärt i antalet element, alltså $O(n^2)$.

$$T(n) = 22 T\left(\frac{n}{3}\right) + O(n^2), \quad T(1) = O(1).$$

- Mästarsatsen med $a = 22$, $b = 3$: $\log_3 22 \approx 2,814 > 2$, så $f(n) = O(n^2) \in O(n^{\log_3 22 - \epsilon})$: fall 1.
- $T(n) \in \Theta(n^{\log_3 22}) \approx \Theta(n^{2,814})$.

Uppgift 2: lösning

- Låt $T(n)$ vara tiden för att multiplicera två $n \times n$ -matriser.
- 22 rekursiva anrop, vart och ett på $\frac{n}{3} \times \frac{n}{3}$ -block: $22 T(n/3)$.
- Att kombinera blocken är bara additioner – linjärt i antalet element, alltså $O(n^2)$.

$$T(n) = 22 T\left(\frac{n}{3}\right) + O(n^2), \quad T(1) = O(1).$$

- Mästarsatsen med $a = 22$, $b = 3$: $\log_3 22 \approx 2,814 > 2$, så $f(n) = O(n^2) \in O(n^{\log_3 22 - \varepsilon})$: fall 1.
- $T(n) \in \Theta(n^{\log_3 22}) \approx \Theta(n^{2,814})$.

Jämfört med Strassen

$\log_3 22 \approx 2,814 > \log_2 7 \approx 2,807$, så 22 multiplikationer hade *inte* slagit Strassen. För det krävs högst 21, eftersom $3^{\log_2 7} \approx 21,8$. Bästa kända för 3×3 är 23 (Laderman, 1976); den bevisade undre gränsen är 19. Om 22 går är fortfarande öppet.

Uppgift 3: Majoritet med dekomposition

Uppgift

Indata är en array A med n element. Konstruera och analysera en algoritm som tar reda på om något element i A är i *majoritet*, det vill säga förekommer mer än $n/2$ gånger, och i så fall returnerar det. Algoritmen ska vara en dekompositionsalgoritm med tidskomplexitet $O(n \log n)$. Enda tillåtna jämförelseoperationen på element i A är $=$.

Uppgift 3: Majoritet med dekomposition

Uppgift

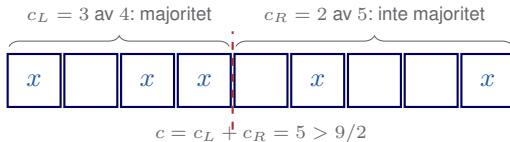
Indata är en array A med n element. Konstruera och analysera en algoritm som tar reda på om något element i A är i *majoritet*, det vill säga förekommer mer än $n/2$ gånger, och i så fall returnerar det. Algoritmen ska vara en dekompositionsalgoritm med tidskomplexitet $O(n \log n)$. Enda tillåtna jämförelseoperationen på element i A är $=$.

- "Mer än $n/2$ gånger" betyder minst $\lfloor n/2 \rfloor + 1$ gånger.
- Ingen ordningsrelation $\Rightarrow$ vi kan inte sortera. (Och ingen hashfunktion heller.)
- Det kan finnas högst *ett* majoritetselement.

Uppgift 3: insikten

Påstående

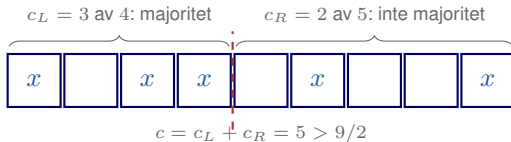
Om x är i majoritet i hela arrayen, så är x i majoritet i åtminstone ena halvan.



Uppgift 3: insikten

Påstående

Om x är i majoritet i hela arrayen, så är x i majoritet i åtminstone ena halvan.

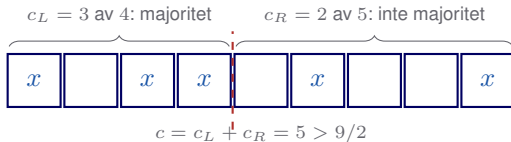


Varför? Låt c_L och c_R vara antalet x i vänstra respektive högra halvan, med storlekar $n_L + n_R = n$. Om x inte vore i majoritet i någon halva skulle $c_L \leq n_L/2$ och $c_R \leq n_R/2$, så $c_L + c_R \leq (n_L + n_R)/2 = n/2$ – men x förekommer mer än $n/2$ gånger. Motsägelse.

Uppgift 3: insikten

Påstående

Om x är i majoritet i hela arrayen, så är x i majoritet i åtminstone ena halvan.



Varför? Låt c_L och c_R vara antalet x i vänstra respektive högra halvan, med storlekar $n_L + n_R = n$. Om x inte vore i majoritet i någon halva skulle $c_L \leq n_L/2$ och $c_R \leq n_R/2$, så $c_L + c_R \leq (n_L + n_R)/2 = n/2$ – men x förekommer mer än $n/2$ gånger. Motsägelse.

Obs! Omvändningen gäller inte: majoritet i en halva ger bara en *kandidat*, som måste kontrolleras.

Uppgift 3: algoritmen

```
1: function MAJORITY( $A[1..n]$ )  $\triangleright$  majoritetselementet eller NULL
2:   if  $n = 1$  then return  $A[1]$ 
3:    $mid \leftarrow \lceil (n + 1)/2 \rceil$ 
4:    $majL \leftarrow$  MAJORITY( $A[1..mid - 1]$ )
5:    $majR \leftarrow$  MAJORITY( $A[mid..n]$ )
6:   if  $majL = majR$  then return  $majL$ 
7:    $cntL \leftarrow 0$ ;  $cntR \leftarrow 0$ 
8:   for  $i \leftarrow 1$  to  $n$  do
9:     if  $A[i] = majL$  then  $cntL \leftarrow cntL + 1$ 
10:    else if  $A[i] = majR$  then  $cntR \leftarrow cntR + 1$ 
11:  if  $2 \cdot cntL > n$  then return  $majL$ 
12:  if  $2 \cdot cntR > n$  then return  $majR$ 
13:  return NULL
```

Rekursiv tanke

- Hitta majoriteten i varje halva rekursivt.
- Det ger högst två kandidater till majoritet i hela arrayen.
- Räkna kandidaternas förekomster i hela arrayen (bara = behövs).
- Returnera den som är i total majoritet, annars NULL.

NULL är inte lika med något element, så en NULL-kandidat räknas aldrig.

Uppgift 3: analys

- Två rekursiva anrop på halva arrayen, följt av efterarbete i $\Theta(n)$ (räkneslingan):

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n), \quad T(1) = \Theta(1).$$

- Mästarsatsen: $\log_2 2 = 1$ och $f(n) = \Theta(n) = \Theta(n^{\log_2 2})$: fall 2.
- $T(n) \in \Theta(n \log n)$, precis som uppgiften kräver.

Uppgift 3: analys

- Två rekursiva anrop på halva arrayen, följt av efterarbete i $\Theta(n)$ (räkneslingan):

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n), \quad T(1) = \Theta(1).$$

- Mästarsatsen: $\log_2 2 = 1$ och $f(n) = \Theta(n) = \Theta(n^{\log_2 2})$: fall 2.
- $T(n) \in \Theta(n \log n)$, precis som uppgiften kräver.

Går det snabbare?

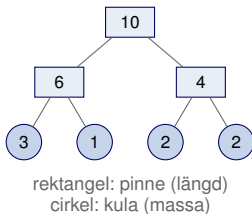
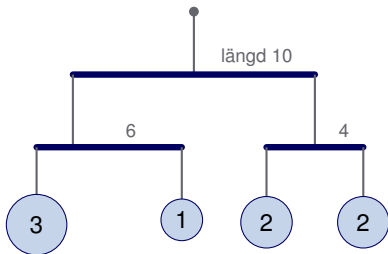
Ja. Boyer–Moore's majoritetsalgoritm hittar den enda möjliga kandidaten i en passage med *en* räknare (öka vid lika, minska annars, byt kandidat vid noll) och verifierar den i en andra passage: $\Theta(n)$ tid, $\Theta(1)$ extra minne, bara = används. Men uppgiften ville ha dekomposition – och $n \log n$ duger.

Uppgift 4: Mobil

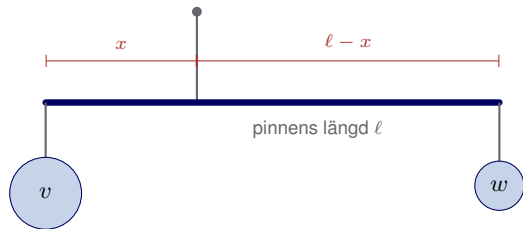
Uppgift

Konstruera en algoritm som balanserar en mobil! Mobilen beskrivs som ett binärträd.

- Löv = kulor som hänger underst.
- Inre noder = tunna raka pinnar; i ändarna hänger sönerna i trådar.
- **Indata:** vikter i noderna. Löv: kulans massa. Inre nod: pinnens längd i cm.
- **Utdata:** samma träd, men varje inre nod anger dessutom hur många cm från pinnens *vänstra* ände upphängningstråden ska fästas.



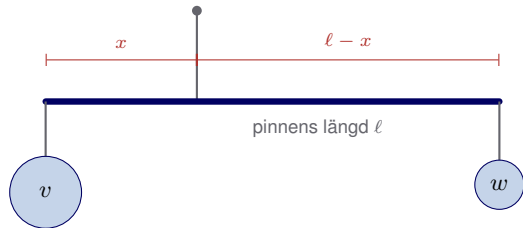
Uppgift 4: lite gymnasiefysik



- Betrakta en pinne av längd ℓ . I vänstra änden hänger vikten v , i högra w .
- Låt x vara avståndet från vänstra änden till upphängningstråden.
- Momentjämvikt kring upphängningspunkten:

$$x v = (\ell - x) w \iff x = \frac{\ell w}{v + w}.$$

Uppgift 4: lite gymnasiefysik



- Betrakta en pinne av längd ℓ . I vänstra änden hänger vikten v , i högra w .
- Låt x vara avståndet från vänstra änden till upphängningstråden.
- Momentjämvikt kring upphängningspunkten:

$$x v = (\ell - x) w \iff x = \frac{\ell w}{v + w}.$$

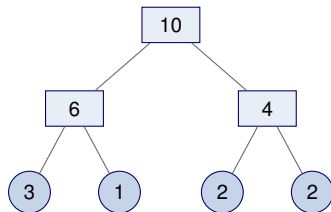
Rekursiv tanke

Vikten i en ände är vikten av *hela* delmobilen under den. Låt därför den rekursiva funktionen returnera delmobilens totala vikt – då kan varje pinne räkna ut sitt x när sönerna är klara. (Pinnar och trådar antas viktlösa.)

Uppgift 4: algoritmen

```
1: function BALANCE( $p$ )           ▷ returnerar delmobilens vikt  
2:   if  $p.left = \text{NULL}$  then return  $p.w$    ▷ lov: kulans massa  
3:    $v_L \leftarrow \text{BALANCE}(p.left)$   
4:    $v_R \leftarrow \text{BALANCE}(p.right)$   
5:    $p.x \leftarrow p.w \cdot v_R / (v_L + v_R)$    ▷  $p.w$  är pinnens längd  
6:   return  $v_L + v_R$ 
```

- Sönerna behandlas före noden själv (postorder): nerifrån och upp.
- Varje nod besöks en gång med konstant arbete: $\Theta(n)$.
- Varje inre nod är ett delproblem; "kombinera" är en formel.

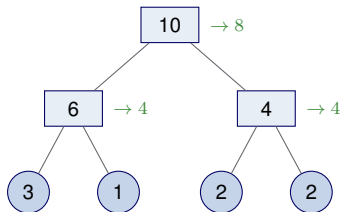


Exempelmobilen från förra bilden.

Uppgift 4: algoritmen

```
1: function BALANCE( $p$ )      ▷ returnerar delmobilens vikt  
2:   if  $p.left = \text{NULL}$  then return  $p.w$   ▷ lov: kulans massa  
3:    $v_L \leftarrow \text{BALANCE}(p.left)$   
4:    $v_R \leftarrow \text{BALANCE}(p.right)$   
5:    $p.x \leftarrow p.w \cdot v_R / (v_L + v_R)$   ▷  $p.w$  är pinnens längd  
6:   return  $v_L + v_R$ 
```

- Sönerna behandlas före noden själv (postorder): nerifrån och upp.
- Varje nod besöks en gång med konstant arbete: $\Theta(n)$.
- Varje inre nod är ett delproblem; "kombinera" är en formel.

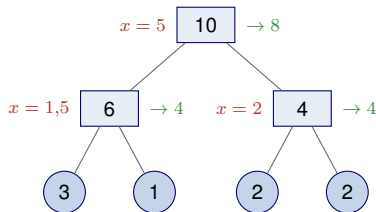


Grönt: vikten som varje anrop returnerar.

Uppgift 4: algoritmen

```
1: function BALANCE( $p$ )           ▷ returnerar delmobilens vikt
2:   if  $p.left = \text{NULL}$  then return  $p.w$    ▷ lov: kulans massa
3:    $v_L \leftarrow \text{BALANCE}(p.left)$ 
4:    $v_R \leftarrow \text{BALANCE}(p.right)$ 
5:    $p.x \leftarrow p.w \cdot v_R / (v_L + v_R)$    ▷  $p.w$  är pinnens längd
6:   return  $v_L + v_R$ 
```

- Sönerna behandlas före noden själv (postorder): nerifrån och upp.
- Varje nod besöks en gång med konstant arbete: $\Theta(n)$.
- Varje inre nod är ett delproblem; "kombinera" är en formel.



Rött: avståndet x i cm från vänstra änden,
t.ex. $6 \cdot \frac{1}{3+1} = 1,5$ för pinnen av längd 6.

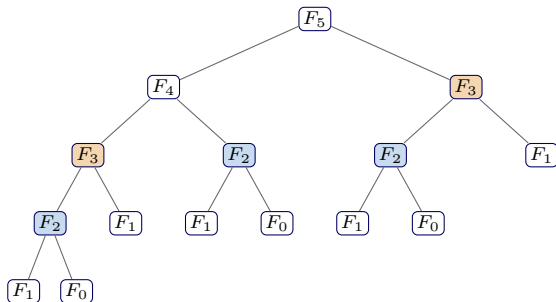
Teori: dynamisk programmering

Varför dynamisk programmering?

Naiv rekursion för Fibonaccital:

```
1: function FIB( $n$ )  
2:   if  $n \leq 1$  then return  $n$   
3:   return FIB( $n - 1$ ) + FIB( $n - 2$ )
```

- F_3 beräknas 2 gånger, F_2 3 gånger, F_1 5 gånger, ...
- Antalet anrop växer som φ^n :
exponentiell tid.
- **Idé:** spara undan delresultaten och beräkna varje delresultat bara en gång.

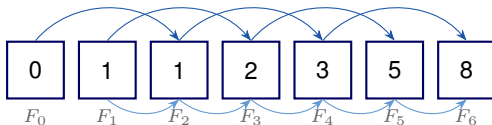


Anropsträdet för FIB(5): samma delproblem (F_3 , F_2) beräknas om och om igen.

Recept för dynamisk programmering

1. **Rekursionsformel.** Hitta en matematisk rekursion som uttrycker lösningen i lösningar till mindre delproblem.
2. **Basfall.** Beskriv de fall som inte behöver rekursion.
3. **Beräkningsordning.** Välj en ordning där allt ett delproblem behöver redan är beräknat – typiskt fylls en tabell ”nerifrån och upp” från basfallen.

Sedan är det bara att implementera. Fundera också på hur mycket av tabellen som faktiskt måste sparas.



FIB nerifrån och upp: $\Theta(n)$ tid, och bara de två senaste talen behöver sparas.

Alternativ: *memoisering* – behåll rekursionen men slå upp i en tabell innan ett delproblem beräknas. Samma delproblem, samma tid, men beräkningsordningen sköter sig själv.

Uppgift 5–6: dynamisk programmering

Uppgift 5: Talföljder

Uppgift

Anta att du har fått rekursionen för en talföljd presenterad för dig. Skriv ned de 7 första talen. Vad behöver man spara under beräkningens gång? Skriv pseudokoden för en dynamisk programmeringsalgorithm för att beräkna S_n .

- a) $S_0 = 1, \quad S_n = 2S_{n-1} + 1$ för $n \geq 1$.
- b) $S_1 = 4, S_2 = 5, \quad S_n = \max(S_{n-1}, S_{n-2}, S_{n-1} - S_{n-2} + 7)$ för $n \geq 3$.
- c) $S_1 = 4, S_2 = 5, \quad S_n = \max(S_{n-2}, S_{n-1} - S_{n-2} + 7)$ för $n \geq 3$.

Uppgift 5: Talföljder

Uppgift

Anta att du har fått rekursionen för en talföljd presenterad för dig. Skriv ned de 7 första talen. Vad behöver man spara under beräkningens gång? Skriv pseudokoden för en dynamisk programmeringsalgorithm för att beräkna S_n .

- a) $S_0 = 1, \quad S_n = 2S_{n-1} + 1$ för $n \geq 1$.
- b) $S_1 = 4, S_2 = 5, \quad S_n = \max(S_{n-1}, S_{n-2}, S_{n-1} - S_{n-2} + 7)$ för $n \geq 3$.
- c) $S_1 = 4, S_2 = 5, \quad S_n = \max(S_{n-2}, S_{n-1} - S_{n-2} + 7)$ för $n \geq 3$.

Tänk på receptet: rekursionsformeln är given – vilka är basfallen, och i vilken ordning räknar vi?

Uppgift 5a

$$S_0 = 1, \quad S_n = 2S_{n-1} + 1 \text{ för } n \geq 1.$$

n	0	1	2	3	4	5	6
S_n	1	3	7	15	31	63	127

- För att beräkna ett tal behöver man bara känna till talet innan: **en** variabel räcker.
- Beräkningsordning: från basfallet och "uppåt".
- Tid $\Theta(n)$, minne $\Theta(1)$.

1: **function** TALFÖLJD-A(n)

2: $v \leftarrow 1$ $\triangleright S_0$

3: **for** $i \leftarrow 1$ **to** n **do**

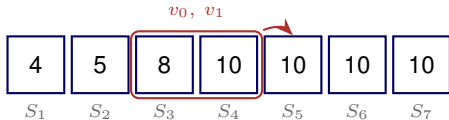
4: $v \leftarrow 2v + 1$ $\triangleright v = S_i$

5: **return** v

(Här råkar det finnas en sluten form, $S_n = 2^{n+1} - 1$, men det är inget vi kan räkna med i allmänhet.)

Uppgift 5b

$S_1 = 4, S_2 = 5,$
 $S_n = \max(S_{n-1}, S_{n-2}, S_{n-1} - S_{n-2} + 7)$ för
 $n \geq 3.$



- Varje tal behöver de **två** föregående: två variabler räcker (ett "glidande fönster").
- Från S_4 är följden konstant: så fort två tal i rad är lika, S , blir nästa $\max(S, S, 7) = S$.

```
1: function TALFÖLJD-B( $n$ )
2:   if  $n = 1$  then return 4
3:   if  $n = 2$  then return 5
4:    $v_0 \leftarrow 4; v_1 \leftarrow 5$   $\triangleright S_{i-2}, S_{i-1}$ 
5:   for  $i \leftarrow 3$  to  $n$  do
6:      $v \leftarrow \max(v_1, v_0, v_1 - v_0 + 7)$ 
7:      $v_0 \leftarrow v_1; v_1 \leftarrow v$ 
8:   return  $v_1$ 
```

OBS: slingan börjar på $i = 3$. Lösningsförslaget i häftet börjar på $i = 2$ och kör därmed ett varv för mycket (TALFÖLJD-B(3) skulle ge $S_4 = 10$ i stället för $S_3 = 8$).

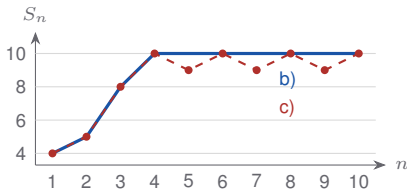
Uppgift 5c

$$S_1 = 4, S_2 = 5,$$
$$S_n = \max(S_{n-2}, S_{n-1} - S_{n-2} + 7) \text{ för } n \geq 3.$$

n	1	2	3	4	5	6	7
S_n	4	5	8	10	9	10	9

- Samma glidande fönster som i b), och samma beräkningsordning.
- Men följden är inte längre monoton: från S_4 alternerar den 10, 9, 10, 9, ... eftersom $\max(10, 9 - 10 + 7) = 10$ och $\max(9, 10 - 9 + 7) = 9$.

```
1: function TALFÖLJD-C( $n$ )
2:   if  $n = 1$  then return 4
3:   if  $n = 2$  then return 5
4:    $v_0 \leftarrow 4; v_1 \leftarrow 5$ 
5:   for  $i \leftarrow 3$  to  $n$  do
6:      $v \leftarrow \max(v_0, v_1 - v_0 + 7)$ 
7:      $v_0 \leftarrow v_1; v_1 \leftarrow v$ 
8:   return  $v_1$ 
```



Uppgift 6a: tvådimensionell rekursion utan indata

Uppgift

Beräkna $M[i, j]$ med dynamisk programmering och argumentera för att beräkningsordningen fungerar:

$$M[i, j] = \begin{cases} 0 & \text{om } i = 0 \text{ eller } j = 0, \\ M[i - 1, j - 1] + i & \text{annars.} \end{cases}$$

- Vi vet inte vad som ska frågas, så vi beräknar hela M , $0 \leq i \leq m$, $0 \leq j \leq n$.

		$\longrightarrow j$				
$i \backslash j$	0	1	2	3	4	
0						
1						
2						
3						
4						

Uppgift 6a: tvådimensionell rekursion utan indata

Uppgift

Beräkna $M[i, j]$ med dynamisk programmering och argumentera för att beräkningsordningen fungerar:

$$M[i, j] = \begin{cases} 0 & \text{om } i = 0 \text{ eller } j = 0, \\ M[i - 1, j - 1] + i & \text{annars.} \end{cases}$$

- Vi vet inte vad som ska frågas, så vi beräknar hela M , $0 \leq i \leq m$, $0 \leq j \leq n$.
- **Basfall:** översta raden och vänstra kolumnen är 0.

		j				
$i \backslash j$		0	1	2	3	4
i	0	0	0	0	0	0
	1	0				
	2	0				
	3	0				
	4	0				

Uppgift 6a: tvådimensionell rekursion utan indata

Uppgift

Beräkna $M[i, j]$ med dynamisk programmering och argumentera för att beräkningsordningen fungerar:

$$M[i, j] = \begin{cases} 0 & \text{om } i = 0 \text{ eller } j = 0, \\ M[i - 1, j - 1] + i & \text{annars.} \end{cases}$$

- Vi vet inte vad som ska frågas, så vi beräknar hela M , $0 \leq i \leq m$, $0 \leq j \leq n$.
- **Basfall:** översta raden och vänstra kolumnen är 0.
- **Beroende:** $M[i, j]$ behöver bara $M[i - 1, j - 1]$ – rutan snett upp till vänster.

$i \backslash j$	0	1	2	3	4
0	0	0	0	0	0
1	0				
2	0			?	
3	0				
4	0				

$M[2, 3]$ behöver $M[1, 2]$, som redan är klar

Uppgift 6a: tvådimensionell rekursion utan indata

Uppgift

Beräkna $M[i, j]$ med dynamisk programmering och argumentera för att beräkningsordningen fungerar:

$$M[i, j] = \begin{cases} 0 & \text{om } i = 0 \text{ eller } j = 0, \\ M[i - 1, j - 1] + i & \text{annars.} \end{cases}$$

- Vi vet inte vad som ska frågas, så vi beräknar hela M , $0 \leq i \leq m$, $0 \leq j \leq n$.
- **Basfall:** översta raden och vänstra kolumnen är 0.
- **Beroende:** $M[i, j]$ behöver bara $M[i - 1, j - 1]$ – rutan snett upp till vänster.
- **Beräkningsordning:** radvis uppifrån, varje rad från vänster till höger.

$i \backslash j$	0	1	2	3	4
0	0	0	0	0	0
1	0				
2	0				
3	0				
4	0				

Uppgift 6a: tvådimensionell rekursion utan indata

Uppgift

Beräkna $M[i, j]$ med dynamisk programmering och argumentera för att beräkningsordningen fungerar:

$$M[i, j] = \begin{cases} 0 & \text{om } i = 0 \text{ eller } j = 0, \\ M[i - 1, j - 1] + i & \text{annars.} \end{cases}$$

- Vi vet inte vad som ska frågas, så vi beräknar hela M , $0 \leq i \leq m$, $0 \leq j \leq n$.
- **Basfall:** översta raden och vänstra kolumnen är 0.
- **Beroende:** $M[i, j]$ behöver bara $M[i - 1, j - 1]$ – rutan snett upp till vänster.
- **Beräkningsordning:** radvis uppifrån, varje rad från vänster till höger.

		$\xrightarrow{j}$				
$i \backslash j$	0	1	2	3	4	
0	0	0	0	0	0	
1	0	1	1	1	1	
2	0	2	3	3	3	
3	0	3	5	6	6	
4	0	4	7	9	10	

Hela M ifylld ($m = n = 4$).

Uppgift 6a: algoritmen

```
1: for  $j \leftarrow 0$  to  $n$  do  $M[0, j] \leftarrow 0$   
2: for  $i \leftarrow 1$  to  $m$  do  
3:    $M[i, 0] \leftarrow 0$   
4:   for  $j \leftarrow 1$  to  $n$  do  
5:      $M[i, j] \leftarrow M[i - 1, j - 1] + i$   
6: return  $M$ 
```

Tiden domineras av den nästlade slingan:
 $\Theta(nm)$.

Varför fungerar ordningen?

- Utom basfallen behöver $M[i, j]$ bara ett värde på raden ovanför (samt radindexet i).
- När vi börjar på rad i är hela rad $i - 1$ redan ifylld, så $M[i - 1, j - 1]$ finns.
- Alltså kan hela M beräknas.

Uppgift 6a: algoritmen

```
1: for  $j \leftarrow 0$  to  $n$  do  $M[0, j] \leftarrow 0$   
2: for  $i \leftarrow 1$  to  $m$  do  
3:    $M[i, 0] \leftarrow 0$   
4:   for  $j \leftarrow 1$  to  $n$  do  
5:      $M[i, j] \leftarrow M[i - 1, j - 1] + i$   
6: return  $M$ 
```

Tiden domineras av den nästlade slingan:
 $\Theta(nm)$.

Varför fungerar ordningen?

- Utom basfallen behöver $M[i, j]$ bara ett värde på raden ovanför (samt radindexet i).
- När vi börjar på rad i är hela rad $i - 1$ redan ifylld, så $M[i - 1, j - 1]$ finns.
- Alltså kan hela M beräknas.

Andra ordningar

Kolumnvis fungerar också, liksom diagonalvis: det enda kravet är att $M[i - 1, j - 1]$ beräknas före $M[i, j]$. Radvis är dessutom minnessnålt om man bara behöver föregående rad.

Uppgift 6b: tvådimensionell rekursion med indata

Uppgift

Givet två strängar a och b av längd n respektive m , låt a_i vara tecknet på position i i a och b_j tecknet på position j i b . Hitta en beräkningsordning och skriv en algoritm som utför följande rekursion med dynamisk programmering:

$$M[i, j] = \begin{cases} 0 & \text{om } i = 0 \text{ eller } j = 0, \\ M[i - 1, j - 1] + 1 & \text{om } a_i = b_j, \\ 0 & \text{annars.} \end{cases}$$

Uppgift 6b: tvådimensionell rekursion med indata

Uppgift

Givet två strängar a och b av längd n respektive m , låt a_i vara tecknet på position i i a och b_j tecknet på position j i b . Hitta en beräkningsordning och skriv en algoritm som utför följande rekursion med dynamisk programmering:

$$M[i, j] = \begin{cases} 0 & \text{om } i = 0 \text{ eller } j = 0, \\ M[i - 1, j - 1] + 1 & \text{om } a_i = b_j, \\ 0 & \text{annars.} \end{cases}$$

- Basfallen är desamma som i a): översta raden och vänstra kolumnen.
- Beroendet är detsamma: $M[i, j]$ behöver bara $M[i - 1, j - 1]$ (och indata a_i, b_j).
- Alltså fungerar samma beräkningsordning: radvis uppifrån, från vänster till höger.

Uppgift 6b: algoritmen

```
1: for  $j \leftarrow 0$  to  $m$  do  $M[0, j] \leftarrow 0$ 
2: for  $i \leftarrow 1$  to  $n$  do
3:    $M[i, 0] \leftarrow 0$ 
4:   for  $j \leftarrow 1$  to  $m$  do
5:     if  $a_i = b_j$  then
6:        $M[i, j] \leftarrow M[i - 1, j - 1] + 1$ 
7:     else  $M[i, j] \leftarrow 0$ 
8: return  $M$ 
```

Tid: $\Theta(nm)$, precis som i a).

Vad är det vi räknar ut?

$M[i, j]$ är längden på det längsta gemensamma suffixet av $a_1 \dots a_i$ och $b_1 \dots b_j$. Största värdet i M är därför längden på den längsta gemensamma delsträngen.

b_j :

		B	A	B	A
	0	0	0	0	0
a_i : A	0	0	1	0	1
B	0	1	0	2	0
A	0	0	2	0	3
B	0	1	0	3	0

$a = \text{ABAB}$, $b = \text{BABA}$.

Längsta gemensamma delsträng: ABA eller BAB (längd 3).

Sammanfattning

Dekomposition

- Dela – lös rekursivt – kombinera. Glöm inte basfallen.
- Skriv ned rekursionsekvationen och lös den med mästarsatsen.
- Ibland ska man räkna operationer exakt (uppgift 1), och då spelar delningen roll.
- Rekursion på träd är dekomposition (uppgift 4).

Dynamisk programmering

- Rekursionsformel – basfall – beräkningsordning.
- Spara delresultaten, beräkna varje delproblem en gång.
- Fundera på hur mycket som faktiskt måste sparas (uppgift 5: två tal).
- I två dimensioner: fyll tabellen så att beroendena redan är klara (uppgift 6: radvis).

Uppgifter och lösningsförslag: Viggo Kann. Upplägg och figuridéer bygger på tidigare övningar av Lisa Li (2019) och Anton Grensjö (2017).